Chapter 9. Multi-agent reinforcement learning

This chapter covers

- Why ordinary Q-learning can fail in the multi-agent setting
- How to deal with the "curse of dimensionality" with multiple agents
- How to implement multi-agent Q-learning models that can perceive other agents
- How to scale multi-agent Q-learning by using the mean field approximation
- How to use DQNs to control dozens of agents in a multi-agent physics simulation and game

So far, the reinforcement learning algorithms we have covered—Q-learning, policy gradients, and actor-critic algorithms—have all been applied to control a single agent in an environment. But what about situations where we want to control multiple agents that can interact with each other? The simplest example of this would be a two-player game where each player is implemented as a reinforcement learning agent. But there are other situations in which we might want to model hundreds or thousands of individual agents all interacting with each other, such as a traffic simulation. In this chapter you will learn how to adapt what you've learned so far into this multi-agent scenario by implementing an algorithm called *mean field Q-learning* (MF-Q), first described in a paper titled "Mean Field Multi-Agent Reinforcement Learning" by Yaodong Yang et al. (2018).

9.1. From one to many agents

In the case of games, the environment might contain other agents that we do not control, often called *non-player characters* (NPCs). For example, in chapter 8 we trained an agent to play Super Mario Bros., which has many NPCs. These NPCs are controlled by some other unseen game logic, but they can and often do interact with the main player. From the perspective of our deep Q-network (DQN) agent, these NPCs are nothing more than